

Name	Siddhanth Kaushik Mungekar
Roll No	23102B0030
Branch	CMPN/B

Code	<pre> # Install only the packages required for this lab packages <- c("readxl", "dplyr", "ggplot2", "cluster", "randomForest", "e1071", "pROC", "plotly") installed <- rownames(installed.packages()) missing <- packages[!(packages %in% installed)] if (length(missing) > 0) { install.packages(missing) } # Load packages lapply(packages, library, character.only = TRUE) cat("All required packages loaded successfully!\n") ... Installing packages into '/usr/local/lib/R/site-library' (as 'lib' is unspecified) also installing the dependencies 'proxy', 'crosstalk' </pre>
------	--

```

# Upload and read the UCI Online Retail dataset
if (requireNamespace("rstudioapi", quietly = TRUE) && rstudioapi::isAvailable()) {
  file_path <- file.choose()
} else {
  file_path <- "/content/Online Retail.xlsx"
}

if (!file.exists(file_path)) {
  stop("Place 'Online Retail.xlsx' in the working directory or select it with file.choose().")
}

retail <- read_excel(file_path)

cat("Rows:", nrow(retail), "\n")
cat("Columns:", ncol(retail), "\n")
head(retail)

```

```

... Rows: 541909
Columns: 8

```

A tibble: 6 × 8

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
<chr>	<chr>	<chr>	<dbl>	<dtm>	<dbl>	<dbl>	<chr>
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom
536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850	United Kingdom
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00	7.65	17850	United Kingdom

```

# Standardize column names
names(retail) <- make.names(names(retail))

# Inspect structure and missing values
str(retail)
colSums(is.na(retail))

# Remove missing CustomerID and duplicate records
retail_clean <- retail %>%
  filter(!is.na(CustomerID)) %>%
  distinct()

# Remove cancellations (InvoiceNo beginning with C)
retail_clean <- retail_clean %>%
  filter(!grepl("^C", InvoiceNo))

# Keep valid quantities and prices
retail_clean <- retail_clean %>%
  filter(Quantity > 0, UnitPrice > 0)

# Create transaction value
retail_clean <- retail_clean %>%
  mutate(
    InvoiceDate = as.POSIXct(InvoiceDate),
    TotalPrice = Quantity * UnitPrice
  )

cat("Rows after preprocessing:", nrow(retail_clean), "\n")
summary(retail_clean[, c("Quantity", "UnitPrice", "TotalPrice")])

```

```

... tibble [541,909 × 8] (S3: tbl_df/tbl/data.frame)
 $ InvoiceNo : chr [1:541909] "536365" "536365" "536365" "536365" ...
 $ StockCode : chr [1:541909] "85123A" "71053" "84406B" "84029G" ...
 $ Description: chr [1:541909] "WHITE HANGING HEART T-LIGHT HOLDER" "WHITE METAL LANTERN" "CREAM CUPID HEARTS COAT HANGER" "KNITTED UNION FLAG HOT WATE
 $ Quantity : num [1:541909] 6 6 8 6 6 2 6 6 6 32 ...
 $ InvoiceDate: POSIXct[1:541909], format: "2010-12-01 08:26:00" "2010-12-01 08:26:00" ...
 $ UnitPrice : num [1:541909] 2.55 3.39 2.75 3.39 3.39 7.65 4.25 1.85 1.85 1.69 ...
 $ CustomerID : num [1:541909] 17850 17850 17850 17850 17850 17850 ...
 $ Country : chr [1:541909] "United Kingdom" "United Kingdom" "United Kingdom" "United Kingdom" ...
 InvoiceNo: 0 StockCode: 0 Description: 1454 Quantity: 0 InvoiceDate: 0 UnitPrice: 0 CustomerID: 135080 Country: 0
 Rows after preprocessing: 307602

```

```

# Reference date = one day after the latest transaction
reference_date <- max(etail_clean$InvoiceDate, na.rm = TRUE) + 86400

customer_data <- etail_clean %>%
  group_by(CustomerID) %>%
  summarise(
    Recency = as.numeric(difftime(reference_date, max(InvoiceDate), units = "days")),
    Frequency = n_distinct(InvoiceNo),
    Monetary = sum(TotalPrice),
    AvgTransactionValue = mean(TotalPrice),
    TotalQuantity = sum(Quantity),
    PurchaseFrequency = Frequency / max(as.numeric(difftime(max(InvoiceDate), min(InvoiceDate), units = "days")), 1),
    .groups = "drop"
  )

summary(customer_data)
head(customer_data)

```

```

...   CustomerID      Recency      Frequency      Monetary
Min.   :12346   Min.    : 1.00   Min.    : 1.000   Min.    :  3.75
1st Qu.:13813   1st Qu.: 18.07   1st Qu.: 1.000   1st Qu.: 306.48
Median :15300   Median : 51.09   Median : 2.000   Median : 668.57
Mean   :15300   Mean    : 93.05   Mean    : 4.272   Mean    :2048.69
3rd Qu.:16779   3rd Qu.:142.73   3rd Qu.: 5.000   3rd Qu.:1660.60
Max.   :18287   Max.    :374.12   Max.    :209.000   Max.    :280206.02
AvgTransactionValue TotalQuantity PurchaseFrequency
Min.    : 2.137   Min.    : 1.0   Min.    : 0.005465
1st Qu.: 12.393   1st Qu.: 159.0   1st Qu.: 0.020279
Median : 17.755   Median : 378.0   Median : 0.046418
Mean    : 68.382   Mean    :1187.6   Mean    : 0.409867
3rd Qu.: 24.879   3rd Qu.: 989.8   3rd Qu.: 1.000000
Max.    :77183.600 Max.    :196915.0 Max.    :26.308436
A tibble: 6 x 7
  CustomerID Recency Frequency Monetary AvgTransactionValue TotalQuantity PurchaseFrequency
  <dbl>      <dbl>      <int>      <dbl>      <dbl>      <dbl>      <dbl>

```

```

# Outlier treatment using IQR capping
cap_outliers <- function(x) {
  q1 <- quantile(x, 0.25, na.rm = TRUE)
  q3 <- quantile(x, 0.75, na.rm = TRUE)
  iqr <- q3 - q1
  lower <- q1 - 1.5 * iqr
  upper <- q3 + 1.5 * iqr
  pmin(pmax(x, lower), upper)
}

feature_cols <- c(
  "Recency", "Frequency", "Monetary",
  "AvgTransactionValue", "TotalQuantity", "PurchaseFrequency"
)

customer_model <- customer_data
customer_model[feature_cols] <- lapply(customer_model[feature_cols], cap_outliers)

# Log transformation reduces skewness
customer_model[feature_cols] <- lapply(
  customer_model[feature_cols],
  function(x) log1p(pmax(x, 0))
)

# Standardization
scaled_features <- scale(customer_model[feature_cols])
rownames(scaled_features) <- customer_model$CustomerID

cat("Scaled feature matrix dimensions:", dim(scaled_features), "\n")

```

```

... Scaled feature matrix dimensions: 4338 6

```

```

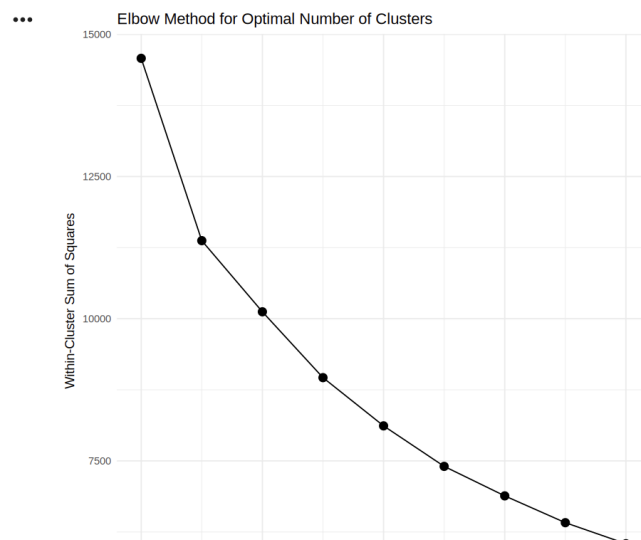
set.seed(123)

k_values <- 2:10
wss <- sapply(k_values, function(k) {
  kmeans(scaled_features, centers = k, nstart = 25)$tot.withinss
})

elbow_df <- data.frame(K = k_values, WSS = wss)

ggplot(elbow_df, aes(K, WSS)) +
  geom_line() +
  geom_point(size = 3) +
  labs(
    title = "Elbow Method for Optimal Number of Clusters",
    x = "Number of Clusters (K)",
    y = "Within-Cluster Sum of Squares"
  ) +
  theme_minimal()

```



```

# Automatically choose an elbow candidate using the largest relative WSS reduction
relative_reduction <- -diff(wss) / head(wss, -1)
optimal_k <- k_values[which.max(relative_reduction) + 1]

cat("Selected K:", optimal_k, "\n")

set.seed(123)
kmeans_model <- kmeans(scaled_features, centers = optimal_k, nstart = 50)

customer_model$KMeansCluster <- factor(kmeans_model$cluster)
table(customer_model$KMeansCluster)

```

```

... Selected K: 3

  1    2    3
1566 1510 1262

```

```
▶ # Sample customers if the dataset is very large
set.seed(123)
sample_n <- min(1500, nrow(scaled_features))
hc_idx <- sample(seq_len(nrow(scaled_features)), sample_n)

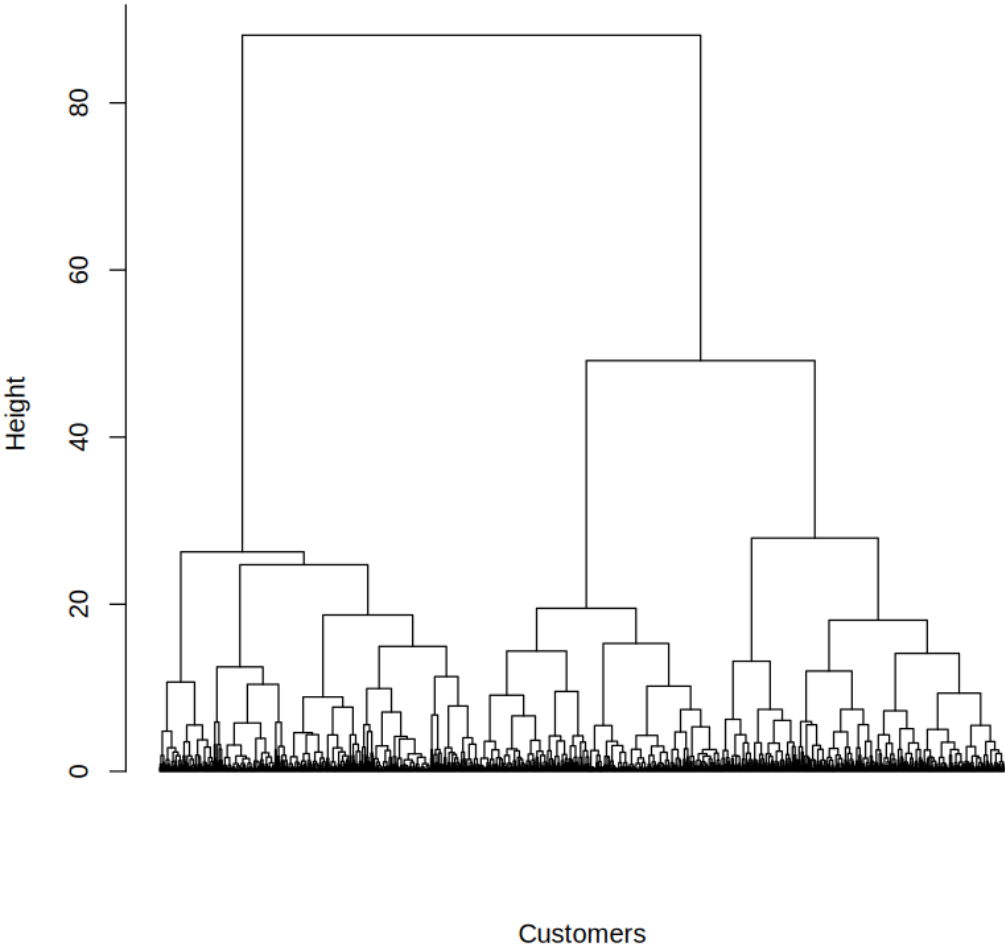
hc_data <- scaled_features[hc_idx, ]
hc_dist <- dist(hc_data)

hc_model <- hclust(hc_dist, method = "ward.D2")

plot(
  hc_model,
  labels = FALSE,
  hang = -1,
  main = "Hierarchical Clustering Dendrogram",
  xlab = "Customers",
  sub = ""
)

hc_clusters <- cutree(hc_model, k = optimal_k)
table(hc_clusters)
```

Hierarchical Clustering Dendrogram



```

# K-Means silhouette
kmeans_sil <- silhouette(kmeans_model$cluster, dist(scaled_features))
kmeans_sil_score <- mean(kmeans_sil[, "sil_width"])

cat("K-Means Average Silhouette Score:", round(kmeans_sil_score, 4), "\n")

# Hierarchical silhouette on the sampled data
hc_sil <- silhouette(hc_clusters, hc_dist)
hc_sil_score <- mean(hc_sil[, "sil_width"])

cat("Hierarchical Average Silhouette Score:", round(hc_sil_score, 4), "\n")

comparison <- data.frame(
  Method = c("K-Means", "Hierarchical Clustering"),
  Silhouette_Score = c(kmeans_sil_score, hc_sil_score)
)
comparison

```

K-Means Average Silhouette Score: 0.3077
 Hierarchical Average Silhouette Score: 0.2999
 A data.frame: 2 × 2

Method	Silhouette_Score
<chr>	<dbl>
K-Means	0.3076666
Hierarchical Clustering	0.2999488

```

▶ pca_model <- prcomp(scaled_features, center = FALSE, scale. = FALSE)

pca_df <- data.frame(
  PC1 = pca_model$x[, 1],
  PC2 = pca_model$x[, 2],
  Cluster = customer_model$KMeansCluster
)

ggplot(pca_df, aes(PC1, PC2, color = Cluster)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Customer Segments using PCA",
    x = "Principal Component 1",
    y = "Principal Component 2"
  ) +
  theme_minimal()

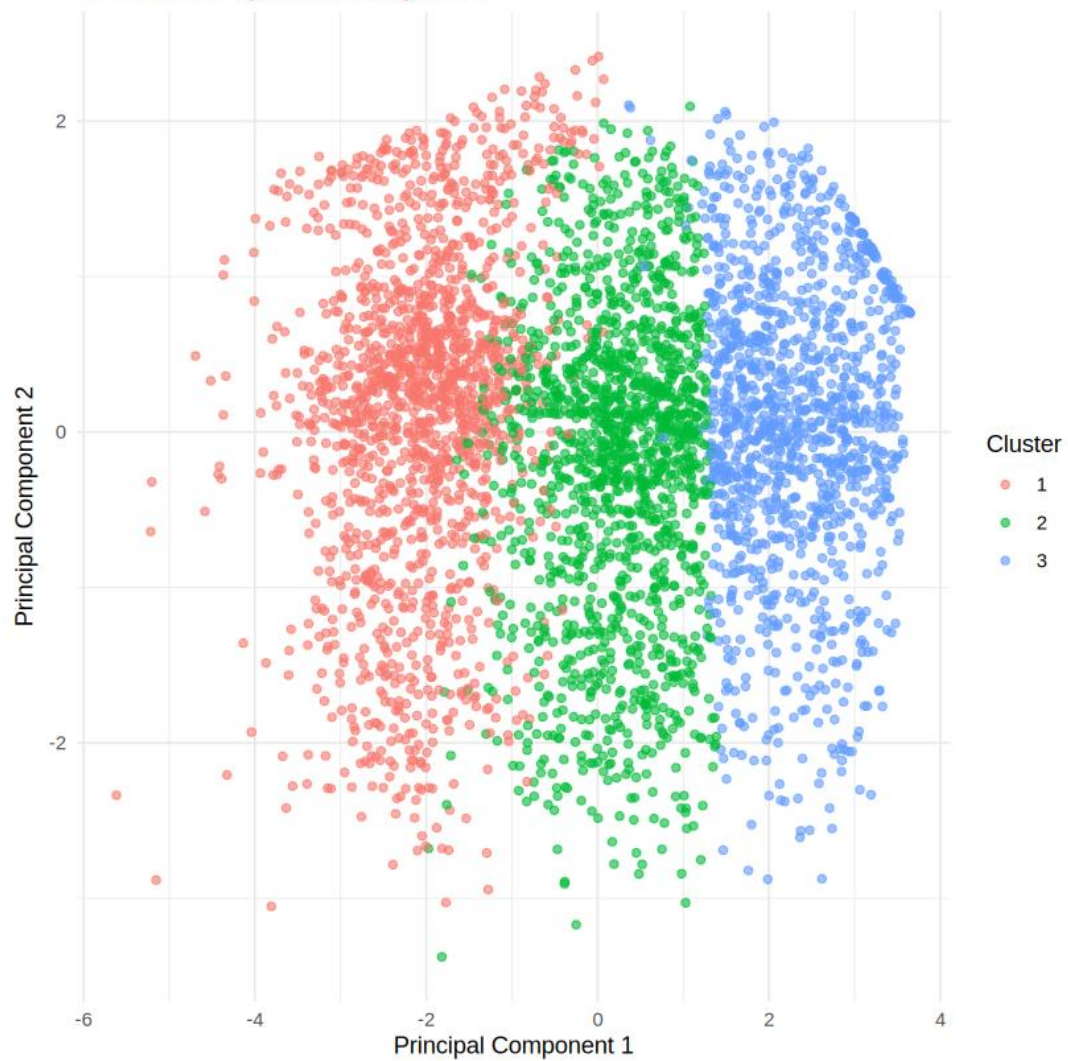
summary(pca_model)

```

... Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6
Standard deviation	1.891	1.0259	0.77105	0.69653	0.46636	0.27197
Proportion of Variance	0.596	0.1754	0.09909	0.08086	0.03625	0.01233
Cumulative Proportion	0.596	0.7715	0.87056	0.95142	0.98767	1.00000

Customer Segments using PCA



```

) profile <- customer_model %>%
  group_by(KMeansCluster) %>%
  summarise(
    Customers = n(),
    AvgRecency = mean(Recency),
    AvgFrequency = mean(Frequency),
    AvgMonetary = mean(Monetary),
    AvgTransactionValue = mean(AvgTransactionValue),
    AvgQuantity = mean(TotalQuantity),
    AvgPurchaseFrequency = mean(PurchaseFrequency),
    .groups = "drop"
  )

```

profile

A tibble: 3 × 8

KMeansCluster	Customers	AvgRecency	AvgFrequency	AvgMonetary	AvgTransactionValue	AvgQuantity	AvgPurchaseFrequency
<fct>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1566	4.653873	0.7164933	5.538094	2.839917	4.892152	0.70565698
2	1510	3.941562	1.3130053	6.463871	2.794443	5.847519	0.03767139
3	1262	2.755979	2.0449360	7.797412	3.040981	7.256953	0.04380727

```

# Ensure KMeansCluster is present in customer_model
if (!"KMeansCluster" %in% names(customer_model)) {
  customer_model$KMeansCluster <- factor(kmeans_model$cluster)
}

# High-value segment = cluster with highest average Monetary value
high_value_cluster <- profile$KMeansCluster[
  which.max(profile$AvgMonetary)
]

cat("High-value cluster:", as.character(high_value_cluster), "\n")

customer_model$HighValue <- ifelse(
  customer_model$KMeansCluster == high_value_cluster, 1, 0
)

table(customer_model$HighValue)

```

```
... High-value cluster: 3
```

```

      0      1
3076 1262

```

```

set.seed(123)

model_df <- customer_model %>%
  select(all_of(feature_cols), HighValue)

# Stratified train/test split
positive <- which(model_df$HighValue == 1)
negative <- which(model_df$HighValue == 0)

train_pos <- sample(positive, floor(0.8 * length(positive)))
train_neg <- sample(negative, floor(0.8 * length(negative)))

train_idx <- c(train_pos, train_neg)
test_idx <- setdiff(seq_len(nrow(model_df)), train_idx)

train <- model_df[train_idx, ]
test <- model_df[test_idx, ]

train$HighValue <- factor(train$HighValue, levels = c(0, 1), labels = c("No", "Yes"))
test$HighValue <- factor(test$HighValue, levels = c(0, 1), labels = c("No", "Yes"))

cat("Training rows:", nrow(train), "\n")
cat("Testing rows:", nrow(test), "\n")
table(train$HighValue)

```

```

Training rows: 3469
Testing rows: 869

```

```

      No      Yes
2460 1009

```

```

set.seed(123)

rf_model <- randomForest(
  HighValue ~ .,
  data = train,
  ntree = 300,
  importance = TRUE
)

rf_pred <- predict(rf_model, newdata = test, type = "response")
rf_prob <- predict(rf_model, newdata = test, type = "prob")[, "Yes"]

rf_cm <- table(
  Actual = test$HighValue,
  Predicted = rf_pred
)

rf_cm

```

	Predicted	
Actual	No	Yes
No	607	9
Yes	10	243

```

set.seed(123)

svm_model <- svm(
  HighValue ~ .,
  data = train,
  kernel = "radial",
  probability = TRUE
)

svm_pred <- predict(svm_model, newdata = test, probability = TRUE)
svm_prob <- attr(svm_pred, "probabilities")[, "Yes"]

svm_cm <- table(
  Actual = test$HighValue,
  Predicted = svm_pred
)

svm_cm

```

	Predicted	
Actual	No	Yes
No	613	3
Yes	0	253

```

calc_metrics <- function(actual, predicted, probability) {
  actual_bin <- ifelse(actual == "Yes", 1, 0)
  pred_bin <- ifelse(predicted == "Yes", 1, 0)

  TP <- sum(actual_bin == 1 & pred_bin == 1)
  TN <- sum(actual_bin == 0 & pred_bin == 0)
  FP <- sum(actual_bin == 0 & pred_bin == 1)
  FN <- sum(actual_bin == 1 & pred_bin == 0)

  accuracy <- (TP + TN) / (TP + TN + FP + FN)
  precision <- ifelse(TP + FP == 0, 0, TP / (TP + FP))
  recall <- ifelse(TP + FN == 0, 0, TP / (TP + FN))
  f1 <- ifelse(precision + recall == 0, 0,
              2 * precision * recall / (precision + recall))

  roc_obj <- roc(actual_bin, probability, quiet = TRUE)
  auc_value <- as.numeric(auc(roc_obj))

  data.frame(
    Accuracy = accuracy,
    Precision = precision,
    Recall = recall,
    F1_Score = f1,
    ROC_AUC = auc_value
  )
}

rf_metrics <- calc_metrics(test$HighValue, rf_pred, rf_prob)
svm_metrics <- calc_metrics(test$HighValue, svm_pred, svm_prob)

results <- rbind(
  Random_Forest = rf_metrics,
  SVM = svm_metrics
)

round(results, 4)

```

```
par(mfrow = c(1, 2))

fourfoldplot(
  rf_cm,
  main = "Random Forest Confusion Matrix"
)

fourfoldplot(
  svm_cm,
  main = "SVM Confusion Matrix"
)

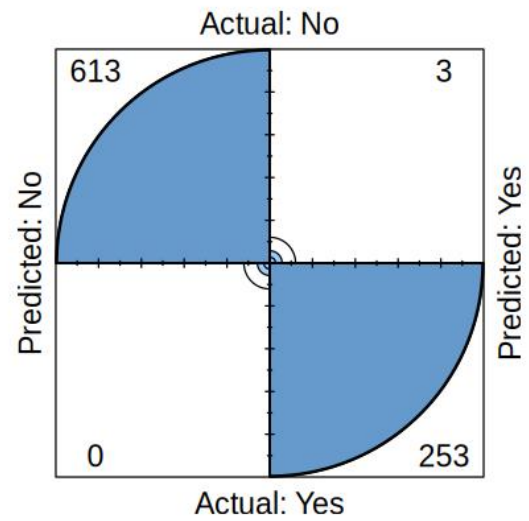
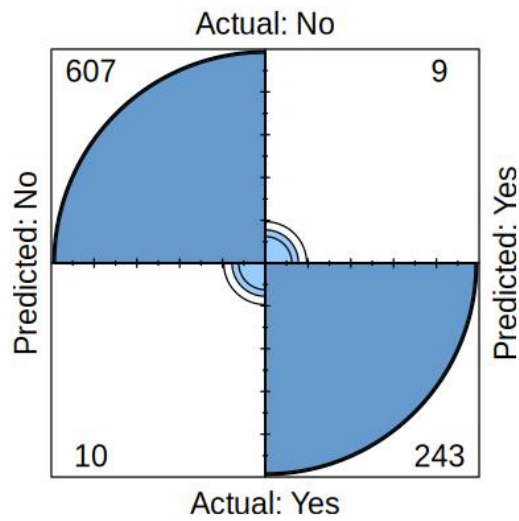
par(mfrow = c(1, 1))

rf_roc <- roc(ifelse(test$HighValue == "Yes", 1, 0), rf_prob, quiet = TRUE)
svm_roc <- roc(ifelse(test$HighValue == "Yes", 1, 0), svm_prob, quiet = TRUE)

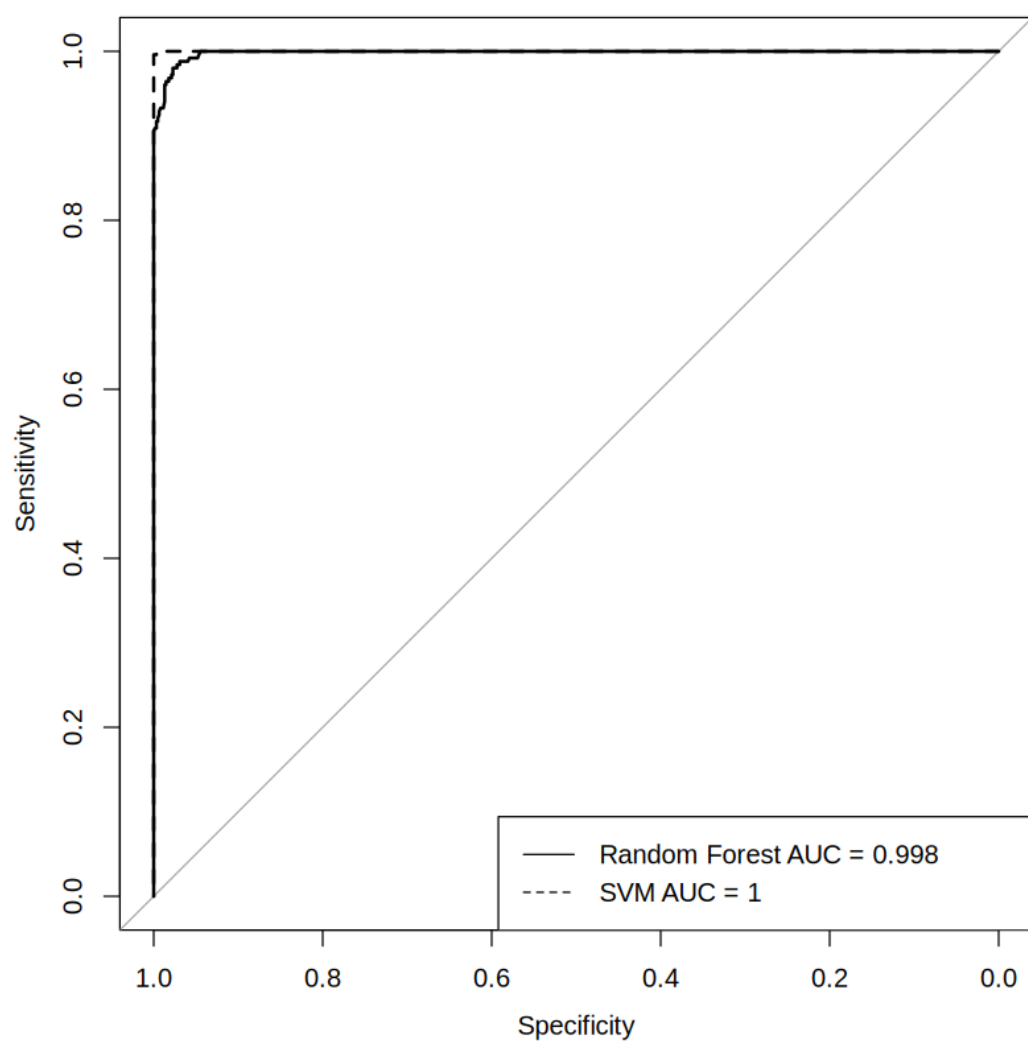
plot(rf_roc, main = "ROC Curves – Random Forest vs SVM")
lines(svm_roc, lty = 2)
legend(
  "bottomright",
  legend = c(
    paste0("Random Forest AUC = ", round(auc(rf_roc), 3)),
    paste0("SVM AUC = ", round(auc(svm_roc), 3))
  ),
  lty = c(1, 2)
)
```

Random Forest Confusion Matrix

SVM Confusion Matrix



ROC Curves – Random Forest vs SVM



```

importance_df <- data.frame(
  Feature = rownames(importance(rf_model)),
  MeanDecreaseGini = importance(rf_model)[, "MeanDecreaseGini"]
) %>%
  arrange(desc(MeanDecreaseGini))

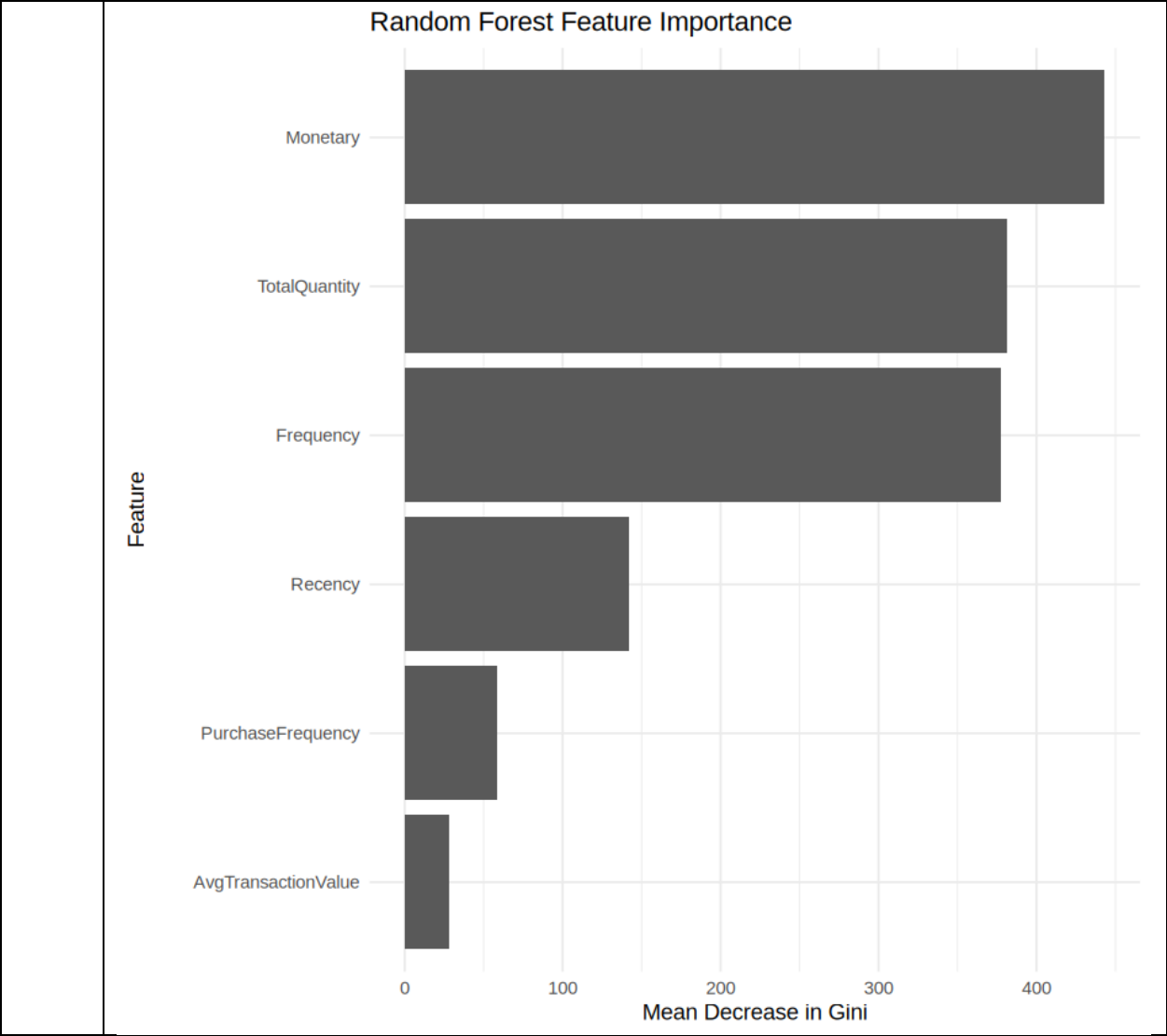
importance_df

ggplot(importance_df, aes(
  x = reorder(Feature, MeanDecreaseGini),
  y = MeanDecreaseGini
)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Random Forest Feature Importance",
    x = "Feature",
    y = "Mean Decrease in Gini"
  ) +
  theme_minimal()

```

A data.frame: 6 × 2

	Feature	MeanDecreaseGini
	<chr>	<dbl>
Monetary	Monetary	443.31239
TotalQuantity	TotalQuantity	381.66300
Frequency	Frequency	377.32151
Recency	Recency	142.30956
PurchaseFrequency	PurchaseFrequency	58.67533
AvgTransactionValue	AvgTransactionValue	28.20372



```
# Check that the data exists
head(pca3_df)
dim(pca3_df)
table(pca3_df$Cluster)
```

```
# Test Plotly
fig <- plot_ly(
  data = pca3_df,
  x = ~PC1,
  y = ~PC2,
  z = ~PC3,
  color = ~Cluster,
  type = "scatter3d",
  mode = "markers",
  marker = list(size = 3)
)
```

```
fig
```

A data.frame: 6 × 4

	PC1	PC2	PC3	Cluster
	<dbl>	<dbl>	<dbl>	<fct>
1	0.01169287	2.41564331	-1.0886834	1
2	3.10931697	0.05734535	0.9321110	3
3	1.69454818	1.63302423	-0.5807347	3
4	-0.12389140	0.68868318	0.8340708	1
5	-2.06232660	0.74959109	-0.5831220	1
6	1.88870521	0.78260414	-0.1465056	3

4338 × 4

```
1 2 3
1566 1510 1262
```